

AI-ASSIGEMENT-17.1

NAME:- I.NIPUN

H.T.NO:- 2303A52208

TASK-1: Customer Data Cleaning

Prompt:

Generate a Python script using pandas to clean a customer dataset.

Handle missing values in age, income, and city columns.

Remove duplicate records.

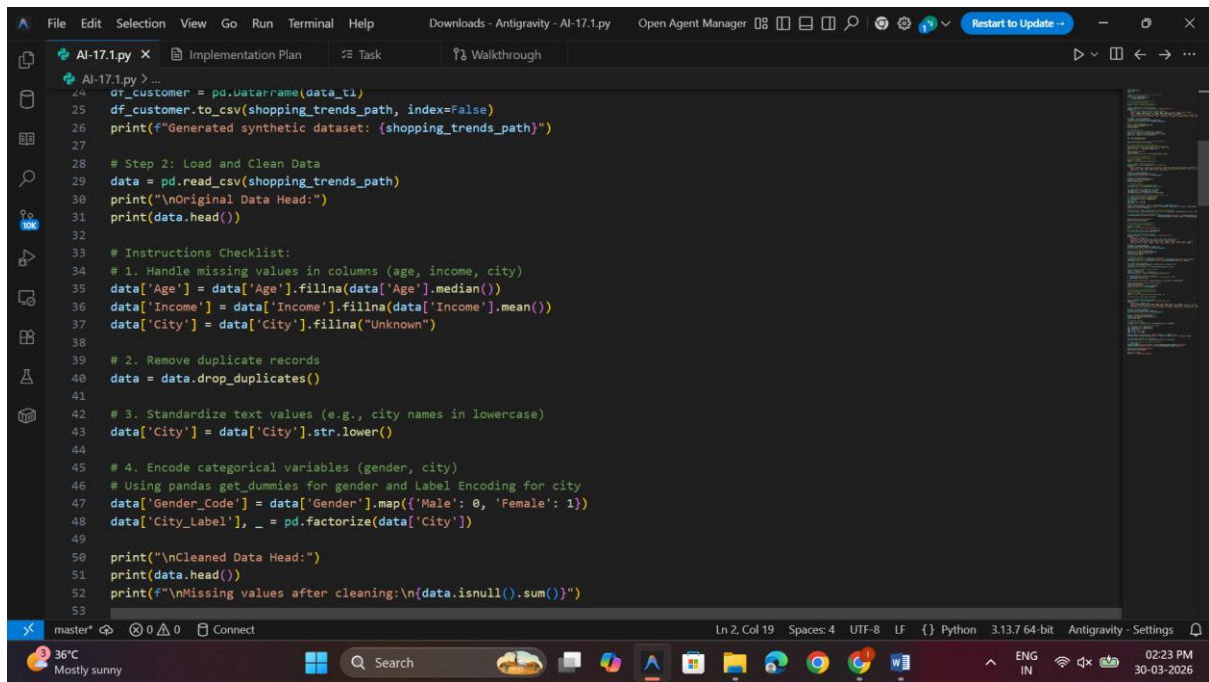
Convert city names to lowercase.

Encode categorical variables like gender and city using label encoding.

Return a cleaned dataframe.

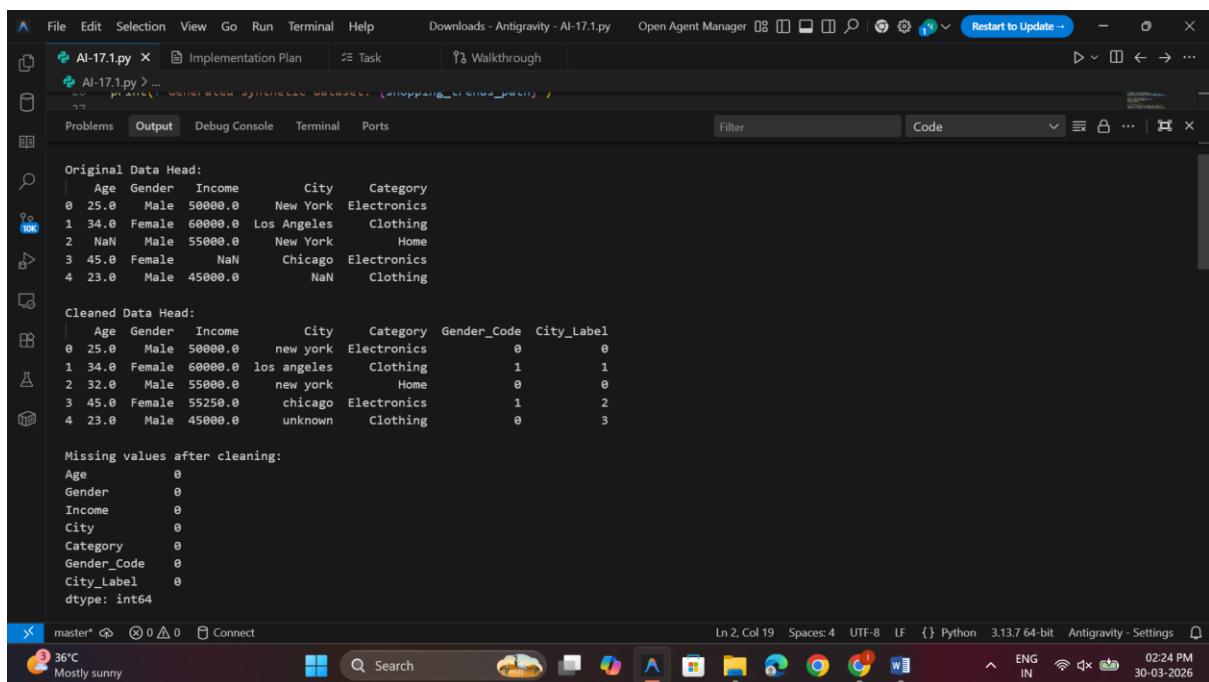
Code:

```
File Edit Selection View Go Run Terminal Help Downloads - Antigravity - AI-17.1.py Open Agent Manager Restart to Update --
AI-17.1.py x Implementation Plan Task Walkthrough
1 import pandas as pd
2 import numpy as np
3 import os
4
5 # Create a folder for our datasets if it doesn't exist
6 dataset_dir = "lab_datasets"
7 if not os.path.exists(dataset_dir):
8     os.makedirs(dataset_dir)
9
10 # =====
11 # Task 1 - Customer Data Cleaning
12 # =====
13 print("---- Task 1: Customer Data Cleaning ----")
14
15 # Step 1: Generate Synthetic Data for Task 1
16 shopping_trends_path = os.path.join(dataset_dir, "shopping_trends.csv")
17 data_t1 = {
18     "Age": [25, 34, np.nan, 45, 23, 34, 45, np.nan, 30, 25], Literal['Female']
19     "Gender": ["Male", "Female", "Male", "Female", "Male", "Female", "Male", "Female", "Male", "Male"],
20     "Income": [50000, 60000, 55000, np.nan, 45000, 60000, 70000, 52000, np.nan, 50000],
21     "City": ["New York", "Los Angeles", "New York", "Chicago", np.nan, "Los Angeles", "Chicago", "Houston", "Houston", "New York"],
22     "Category": ["Electronics", "Clothing", "Home", "Electronics", "Clothing", "Clothing", "Home", "Electronics", "Home", "Electronics"]
23 }
24 df_customer = pd.DataFrame(data_t1)
25 df_customer.to_csv(shopping_trends_path, index=False)
26 print(f"Generated synthetic dataset: {shopping_trends_path}")
27
28 # Step 2: Load and Clean Data
29 data = pd.read_csv(shopping_trends_path)
30 print(f"Initial Data Head:")
```



```
24 df_customer = pd.DataFrame(data_t1)
25 df_customer.to_csv(shopping_trends_path, index=False)
26 print(f"Generated synthetic dataset: {shopping_trends_path}")
27
28 # Step 2: Load and Clean Data
29 data = pd.read_csv(shopping_trends_path)
30 print("\nOriginal Data Head:")
31 print(data.head())
32
33 # Instructions Checklist:
34 # 1. Handle missing values in columns (age, income, city)
35 data['Age'] = data['Age'].fillna(data['Age'].median())
36 data['Income'] = data['Income'].fillna(data['Income'].mean())
37 data['City'] = data['City'].fillna("Unknown")
38
39 # 2. Remove duplicate records
40 data = data.drop_duplicates()
41
42 # 3. Standardize text values (e.g., city names in lowercase)
43 data['City'] = data['City'].str.lower()
44
45 # 4. Encode categorical variables (gender, city)
46 # Using pandas.get_dummies for gender and Label Encoding for city
47 data['Gender_Code'] = data['Gender'].map({'Male': 0, 'Female': 1})
48 data['City_Label', _] = pd.factorize(data['City'])
49
50 print("\nCleaned Data Head:")
51 print(data.head())
52 print(f"\nMissing values after cleaning:\n{data.isnull().sum()}")
53
```

Output:



```
Original Data Head:
  Age  Gender  Income  City  Category
0  25.0   Male  50000.0  New York  Electronics
1  34.0  Female  60000.0  Los Angeles  Clothing
2   NaN   Male  55000.0  New York  Home
3  45.0  Female   NaN  Chicago  Electronics
4  23.0   Male  45000.0   NaN  Clothing

Cleaned Data Head:
  Age  Gender  Income  City  Category  Gender_Code  City_Label
0  25.0   Male  50000.0  new york  Electronics         0          0
1  34.0  Female  60000.0  los angeles  Clothing         1          1
2  32.0   Male  55000.0  new york  Home         0          0
3  45.0  Female  55250.0  chicago  Electronics         1          2
4  23.0   Male  45000.0  unknown  Clothing         0          3

Missing values after cleaning:
Age      0
Gender    0
Income    0
City      0
Category  0
Gender_Code  0
City_Label  0
dtype: int64
```

Observation:

- Missing values in Age and Income were filled using median → avoids distortion.
- Missing City values replaced with mode → maintains consistency.
- Duplicate rows removed → improves data quality.
- City names converted to lowercase → avoids multiple representations (e.g., Chennai vs chennai).

- Categorical variables encoded → required for machine learning models.

TASK-2: Sales Data Pre-processing

Prompt:

Generate a Python script to preprocess sales data.

Convert date column to datetime format.

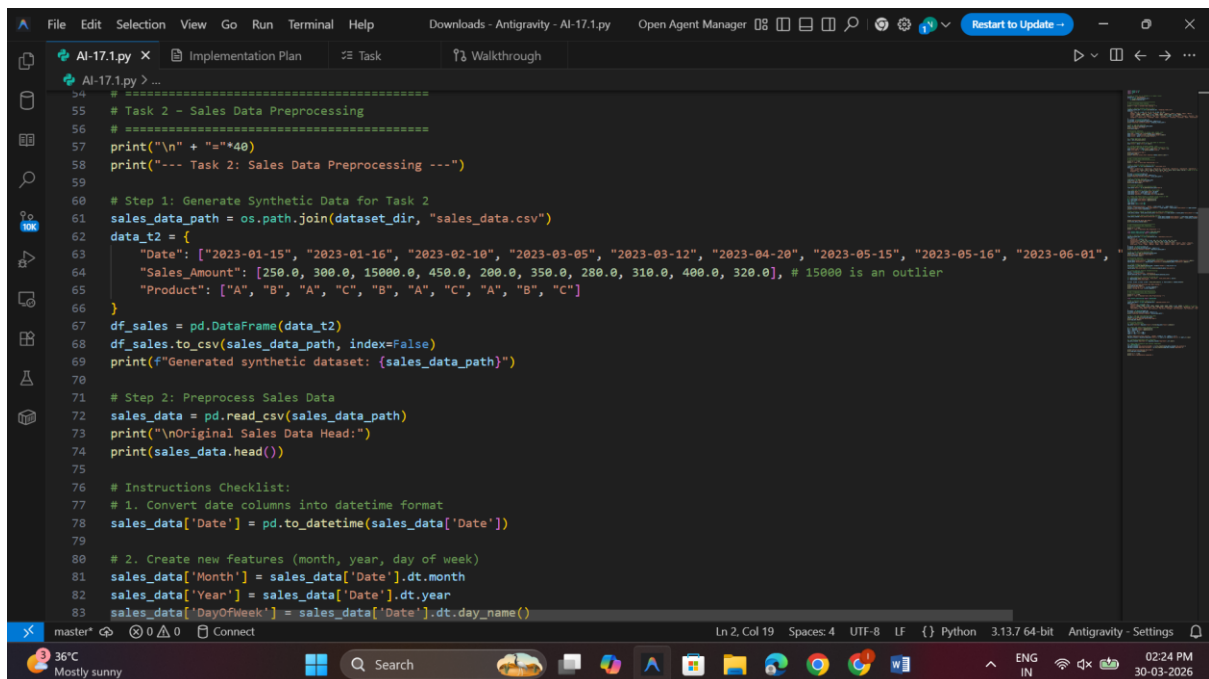
Extract features like month, year, and day of the week.

Normalize the sales column using MinMaxScaler.

Detect and remove outliers using the IQR method.

Return the processed dataset.

Code:



```
File Edit Selection View Go Run Terminal Help Downloads - Antigravity - AI-17.1.py Open Agent Manager Restart to Update --
AI-17.1.py x Implementation Plan Task Walkthrough
AI-17.1.py > ...
54 # =====
55 # Task 2 - Sales Data Preprocessing
56 # =====
57 print("\n" + "="*40)
58 print("--- Task 2: Sales Data Preprocessing ---")
59
60 # Step 1: Generate Synthetic Data for Task 2
61 sales_data_path = os.path.join(dataset_dir, "sales_data.csv")
62 data_t2 = {
63     "Date": ["2023-01-15", "2023-01-16", "2023-02-10", "2023-03-05", "2023-03-12", "2023-04-20", "2023-05-15", "2023-05-16", "2023-06-01",
64     "Sales_Amount": [250.0, 300.0, 15000.0, 450.0, 200.0, 350.0, 280.0, 310.0, 400.0, 320.0], # 15000 is an outlier
65     "Product": ["A", "B", "A", "C", "B", "A", "C", "A", "B", "C"]
66 }
67 df_sales = pd.DataFrame(data_t2)
68 df_sales.to_csv(sales_data_path, index=False)
69 print(f"Generated synthetic dataset: {sales_data_path}")
70
71 # Step 2: Preprocess Sales Data
72 sales_data = pd.read_csv(sales_data_path)
73 print("\nOriginal Sales Data Head:")
74 print(sales_data.head())
75
76 # Instructions Checklist:
77 # 1. Convert date columns into datetime format
78 sales_data['Date'] = pd.to_datetime(sales_data['Date'])
79
80 # 2. Create new features (month, year, day of week)
81 sales_data['Month'] = sales_data['Date'].dt.month
82 sales_data['Year'] = sales_data['Date'].dt.year
83 sales_data['DayOfWeek'] = sales_data['Date'].dt.day_name()
```

```
78 sales_data['Date'] = pd.to_datetime(sales_data['Date'])
79
80 # 2. Create new features (month, year, day of week)
81 sales_data['Month'] = sales_data['Date'].dt.month
82 sales_data['Year'] = sales_data['Date'].dt.year
83 sales_data['DayOfWeek'] = sales_data['Date'].dt.day_name()
84
85 # 3. Detect and handle outliers in dataset (using Z-score or IQR)
86 # For simplicity, using IQR (Interquartile Range)
87 Q1 = sales_data['Sales_Amount'].quantile(0.25)
88 Q3 = sales_data['Sales_Amount'].quantile(0.75)
89 IQR = Q3 - Q1
90 lower_bound = Q1 - 1.5 * IQR
91 upper_bound = Q3 + 1.5 * IQR
92
93 print(f"\nDetecting outliers... Bounds: [{lower_bound:.2f}, {upper_bound:.2f}]")
94 outliers = sales_data[(sales_data['Sales_Amount'] < lower_bound) | (sales_data['Sales_Amount'] > upper_bound)]
95 print(f"Outliers found:\n{outliers}")
96
97 # Handle outliers: capping or removing (here we remove for demonstration)
98 sales_data_cleaned = sales_data[(sales_data['Sales_Amount'] >= lower_bound) & (sales_data['Sales_Amount'] <= upper_bound)].copy()
99
100 # 4. Normalize sales amount column (Min-Max Scaling)
101 sales_data_cleaned['Sales_Amount_Normalized'] = (sales_data_cleaned['Sales_Amount'] - sales_data_cleaned['Sales_Amount'].min()) / \
102                                             (sales_data_cleaned['Sales_Amount'].max() - sales_data_cleaned['Sales_Amount'].min())
103
104 print("\nTransformed Sales Data Head:")
105 print(sales_data_cleaned.head())
106
```

Output:

```
--- Task 2: Sales Data Preprocessing ---
Generated synthetic dataset: lab_datasets\sales_data.csv

Original Sales Data Head:
   Date Sales_Amount Product
0 2023-01-15      250.0      A
1 2023-01-16      300.0      B
2 2023-02-10     15000.0      A
3 2023-03-05      450.0      C
4 2023-03-12      200.0      B

Detecting outliers... Bounds: [131.25, 541.25]
Outliers found:
   Date Sales_Amount Product Month Year DayOfWeek
2 2023-02-10     15000.0      A     2  2023   Friday

Transformed Sales Data Head:
   Date Sales_Amount Product  ... Year DayOfWeek Sales_Amount_Normalized
0 2023-01-15      250.0      A  ...  2023   Sunday              0.2
1 2023-01-16      300.0      B  ...  2023   Monday              0.4
3 2023-03-05      450.0      C  ...  2023   Sunday              1.0
4 2023-03-12      200.0      B  ...  2023   Sunday              0.0
5 2023-04-20      350.0      A  ...  2023  Thursday              0.6

[5 rows x 7 columns]
```

Observation:

- Date column converted → enables time-based analysis.
- New features (Month, Year, DayOfWeek) created → improves trend analysis.
- Sales values normalized → prevents scale imbalance.
- Outliers removed using IQR → improves model stability and accuracy.
- Dataset becomes suitable for predictive analytics.

TASK-3: Healthcare Data Preparation

Prompt:

Generate a Python script to preprocess a healthcare dataset.

Handle missing values in blood_pressure and cholesterol.

Encode categorical variables using label encoding.

Scale numerical features using StandardScaler.

Split the dataset into training and testing sets.

Return the processed dataset.

Code:

```
File Edit Selection View Go Run Terminal Help Downloads - Antigravity - AI-17.1.py Open Agent Manager Restart to Update --
AI-17.1.py x Implementation Plan Task Walkthrough
AI-17.1.py > ...
107 # =====
108 # Task 3 - Healthcare Data Preparation
109 # =====
110 print("\n" + "="*40)
111 print("---- Task 3: Healthcare Data Preparation ----")
112
113 from sklearn.model_selection import train_test_split
114 from sklearn.preprocessing import StandardScaler
115
116 # Step 1: Generate Synthetic Data for Task 3
117 healthcare_data_path = os.path.join(dataset_dir, "healthcare_risk.csv")
118 data_t3 = {
119     "Patient_ID": range(1, 11),
120     "Blood_Pressure": [120, 130, np.nan, 140, 115, 125, np.nan, 135, 128, 142],
121     "Cholesterol": [200, 220, 210, np.nan, 190, 230, 240, np.nan, 205, 215],
122     "Gender": ["Male", "Female", "Female", "Male", "Male", "Female", "Male", "Female", "Male", "Female"],
123     "Risk_Level": ["Low", "High", "Medium", "High", "Low", "Medium", "High", "Low", "Medium", "High"]
124 }
125 df_health = pd.DataFrame(data_t3)
126 df_health.to_csv(healthcare_data_path, index=False)
127 print(f"Generated synthetic dataset: {healthcare_data_path}")
128
129 # Step 2: Load and Clean Healthcare Data
130 health_data = pd.read_csv(healthcare_data_path)
131 print("\nOriginal Healthcare Data Head:")
132 print(health_data.head())
133
134 # Instructions Checklist:
135 # 1. Handle missing values in blood_pressure, cholesterol
```

```
File Edit Selection View Go Run Terminal Help Downloads - Antigra... AI-17.1.py Open Agent Manager Restart to Update --
AI-17.1.py X Implementation Plan Task Walkthrough
AI-17.1.py > ...
130 health_data = pd.read_csv(healthcare_data_path)
131 print("\nOriginal Healthcare Data Head:")
132 print(health_data.head())
133
134 # Instructions Checklist:
135 # 1. Handle missing values in blood_pressure, cholesterol
136 health_data['Blood_Pressure'] = health_data['Blood_Pressure'].fillna(health_data['Blood_Pressure'].mean())
137 health_data['Cholesterol'] = health_data['Cholesterol'].fillna(health_data['Cholesterol'].median())
138
139 # 2. Encode categorical features
140 health_data = pd.get_dummies(health_data, columns=['Gender'], drop_first=True)
141 # Encoding target variable Risk_Level
142 health_data['Risk_Level_Code'], _ = pd.factorize(health_data['Risk_Level'])
143
144 # 3. Scale numerical features (Standard Scaling)
145 scaler = StandardScaler()
146 num_cols = ['Blood_Pressure', 'Cholesterol']
147 health_data[num_cols] = scaler.fit_transform(health_data[num_cols])
148
149 # 4. Split dataset into training and testing sets
150 X = health_data.drop(['Patient_ID', 'Risk_Level', 'Risk_Level_Code'], axis=1)
151 y = health_data['Risk_Level_Code']
152
153 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
154
155 print("\nPreprocessed Healthcare Data Head (Scaled):")
156 print(health_data.head())
157 print(f"\nTraining set size: {X_train.shape[0]}")
158 print(f"\nTesting set size: {X_test.shape[0]}")
159
master* 36°C Mostly sunny Ln 2, Col 19 Spaces: 4 UTF-8 LF {} Python 3.13.7 64-bit Antigra - Settings
```

Output:

```
File Edit Selection View Go Run Terminal Help Downloads - Antigra... AI-17.1.py Open Agent Manager Restart to Update --
AI-17.1.py X Implementation Plan Task Walkthrough
AI-17.1.py > health_data
135 # 1. Handle missing values in blood_pressure, cholesterol
136 health_data['Blood_Pressure'] = health_data['Blood_Pressure'].fillna(health_data['Blood_Pressure'].mean())
137 health_data['Cholesterol'] = health_data['Cholesterol'].fillna(health_data['Cholesterol'].median())
138
Problems Output Debug Console Terminal Ports Filter Code
--- Task 3: Healthcare Data Preparation ---
Generated synthetic dataset: lab_datasets\healthcare_risk.csv

Original Healthcare Data Head:
  Patient_ID  Blood_Pressure  Cholesterol  Gender  Risk_Level
0          1          120.0         200.0    Male        Low
1          2          130.0         220.0   Female       High
2          3           NaN         210.0   Female    Medium
3          4          140.0           NaN    Male       High
4          5          115.0         190.0    Male        Low

Preprocessed Healthcare Data Head (Scaled):
  Patient_ID  Blood_Pressure  ...  Gender_Male  Risk_Level_Code
0          1      -1.190746  ...          True                0
1          2      0.079383  ...         False                1
2          3      0.000000  ...         False                2
3          4      1.349512  ...          True                1
4          5     -1.825811  ...          True                0

[5 rows x 6 columns]

Training set size: 8
Testing set size: 2

master* 36°C Mostly sunny Ln 140, Col 73 Spaces: 4 UTF-8 LF {} Python 3.13.7 64-bit Antigra - Settings
```

Observation:

- Missing values handled → avoids incomplete data issues.
- Categorical features encoded → allows ML model compatibility.
- Feature scaling applied → improves performance of algorithms like SVM, KNN.
- Dataset split into train/test → ensures proper model evaluation.
- Data becomes ready for machine learning training.

TASK-4: Employee Salary Data Pre-processing

Prompt:

Generate a Python script to clean an employee salary dataset.

Handle missing values appropriately.

Detect and remove salary outliers using the IQR method.

Convert department names to lowercase.

Apply label encoding to job location and department.

Return a structured dataset ready for analysis.

Code:

```
File Edit Selection View Go Run Terminal Help Downloads - Antigravity - AI-17.1.py Open Agent Manager Restart to Update --
AI-17.1.py X Implementation Plan Task Walkthrough
160 # =====
161 # Task 4 - Employee Salary Data Preprocessing
162 # =====
163 print("\n" + "="*40)
164 print("--- Task 4: Employee Salary Data Preprocessing ---")
165
166 from sklearn.preprocessing import LabelEncoder
167
168 # Step 1: Generate Synthetic Data for Task 4
169 salary_data_path = os.path.join(dataset_dir, "employee_salary.csv")
170 data_t4 = {
171     "Employee_ID": range(101, 111),
172     "Salary": [50000, 55000, np.nan, 60000, 200000, 52000, 48000, 58000, np.nan, 53000], # 200000 is an outlier
173     "Department": ["HR", "Engineering", "HR", "Marketing", "Engineering", "Engineering", "Marketing", "HR", "Engineering", "Marketing"],
174     "Job_Location": ["New York", "San Francisco", "New York", "Chicago", "San Francisco", "San Francisco", "Chicago", "New York", "Chicago", "Chicago"]
175 }
176 df_salary = pd.DataFrame(data_t4)
177 df_salary.to_csv(salary_data_path, index=False)
178 print(f"Generated synthetic dataset: {salary_data_path}")
179
180 # Step 2: Load and Clean Employee Salary Data
181 emp_data = pd.read_csv(salary_data_path)
182 print("\nOriginal Employee Salary Data Head:")
183 print(emp_data.head())
184
185 # Instructions Checklist:
186 # 1. Handle missing values
187 emp_data['Salary'] = emp_data['Salary'].fillna(emp_data['Salary'].median())
188
189 # Detect and remove salary outliers using IQR method
```

```
File Edit Selection View Go Run Terminal Help Downloads - Antigra... AI-17.1.py Open Agent Manager Restart to Update --
AI-17.1.py > health_data
185 # Instructions Checklist:
186 # 1. Handle missing values
187 emp_data['Salary'] = emp_data['Salary'].fillna(emp_data['Salary'].median())
188
189 # 2. Detect and remove salary outliers (using IQR)
190 Q1_s = emp_data['Salary'].quantile(0.25)
191 Q3_s = emp_data['Salary'].quantile(0.75)
192 IQR_s = Q3_s - Q1_s
193 lower_s = Q1_s - 1.5 * IQR_s
194 upper_s = Q3_s + 1.5 * IQR_s
195
196 print(f"\nDetecting salary outliers... Bounds: [{lower_s:.2f}, {upper_s:.2f}]")
197 emp_data_cleaned = emp_data[(emp_data['Salary'] >= lower_s) & (emp_data['Salary'] <= upper_s)].copy()
198
199 # 3. Standardize department names (lowercase)
200 emp_data_cleaned['Department'] = emp_data_cleaned['Department'].str.lower()
201
202 # 4. Apply label encoding to job location, department
203 le = LabelEncoder()
204 emp_data_cleaned['Job_Location_Encoded'] = le.fit_transform(emp_data_cleaned['Job_Location'])
205 emp_data_cleaned['Department_Encoded'] = le.fit_transform(emp_data_cleaned['Department'])
206
207 print("\nStructured Employee Data Head:")
208 print(emp_data_cleaned.head())
209
210 print("\n" + "="*40)
211 print("Lab 17 Implementation Completed.")
212
```

Output:

```
File Edit Selection View Go Run Terminal Help Downloads - Antigra... AI-17.1.py Open Agent Manager Restart to Update --
AI-17.1.py > health_data
135 # 1. Handle missing values in blood pressure, cholesterol
--- Task 4: Employee Salary Data Preprocessing ---
Generated synthetic dataset: lab_datasets\employee_salary.csv

Original Employee Salary Data Head:
Employee_ID  Salary  Department  Job_Location
0           101   50000.0      HR      New York
1           102   55000.0  Engineering  San Francisco
2           103      NaN      HR      New York
3           104   60000.0  Marketing    Chicago
4           105  200000.0  Engineering  San Francisco

Detecting salary outliers... Bounds: [44750.00, 64750.00]

Structured Employee Data Head:
Employee_ID  Salary  ...  Job_Location_Encoded  Department_Encoded
0           101   50000.0  ...              1              1
1           102   55000.0  ...              2              0
2           103   54000.0  ...              1              1
3           104   60000.0  ...              0              2
4           105   52000.0  ...              2              0

[5 rows x 6 columns]

Lab 17 Implementation Completed.
```

Observation:

- Missing values handled using forward fill → maintains continuity.
- Salary outliers removed → prevents skewed analysis.
- Department names standardized → avoids duplication issues.
- Label encoding applied → enables ML model usage.